# Logistic regression và softmax

Dự đoán xác suất để phân loại

Phase 2 · Bài 2.3

# Bộ lọc thư rác

“Chúc mừng! Đã trúng iPhone, bấm vào đây để nhận.”

Bộ lọc không cần biết chắc. Nó cần nói: **xác suất spam là 0.97**.

Người dùng tự chọn chặn từ mức nào.

CÂU HỎI

Linear regression cho một số thực bất kỳ.

Làm sao biến nó thành xác suất, và học nó bằng hàm mất mát nào?

# Mục tiêu sau buổi này

- Chỉ ra vì sao đầu ra linear regression chưa phải xác suất; đọc hệ số logistic regression theo odds.
- Suy ra gradient của cross-entropy, tự cài bằng NumPy, kiểm gradient, đối chiếu scikit-learn.
- Chọn ngưỡng quyết định theo cái giá của từng loại sai.
- Cài softmax không tràn số cho bài toán nhiều lớp.

# Dữ liệu

Mỗi email: **số link** và **số từ quảng cáo** (“miễn phí”, “trúng thưởng”).

Nhãn 1 là spam, 0 là thư thường.

400 email: 300 để học, 100 để kiểm tra. Khoảng một phần ba là spam.

```
z_true = 1.1 * links + 0.9 * kw - 4.5
y = rng.random(n) < 1 / (1 + np.exp(-z_true))
X = np.column_stack([links, kw])     # (400, 2)
```

Tự sinh từ quy luật đã biết để đối chiếu xem mô hình học được gì.

01

# Đầu ra linear regression và xác suất

# Đầu ra không phải xác suất

```
lin = LinearRegression().fit(X_tr, y_tr)
p_lin = lin.predict(X_tr)
```

KẾT QUẢ

```
dự đoán nhỏ nhất: -0.2  lớn nhất: 1.34
số email có 'xác suất' ngoài [0, 1]: 54
```

"Xác suất $-0.2$" không có nghĩa gì.

Và còn một vấn đề nghiêm trọng hơn.

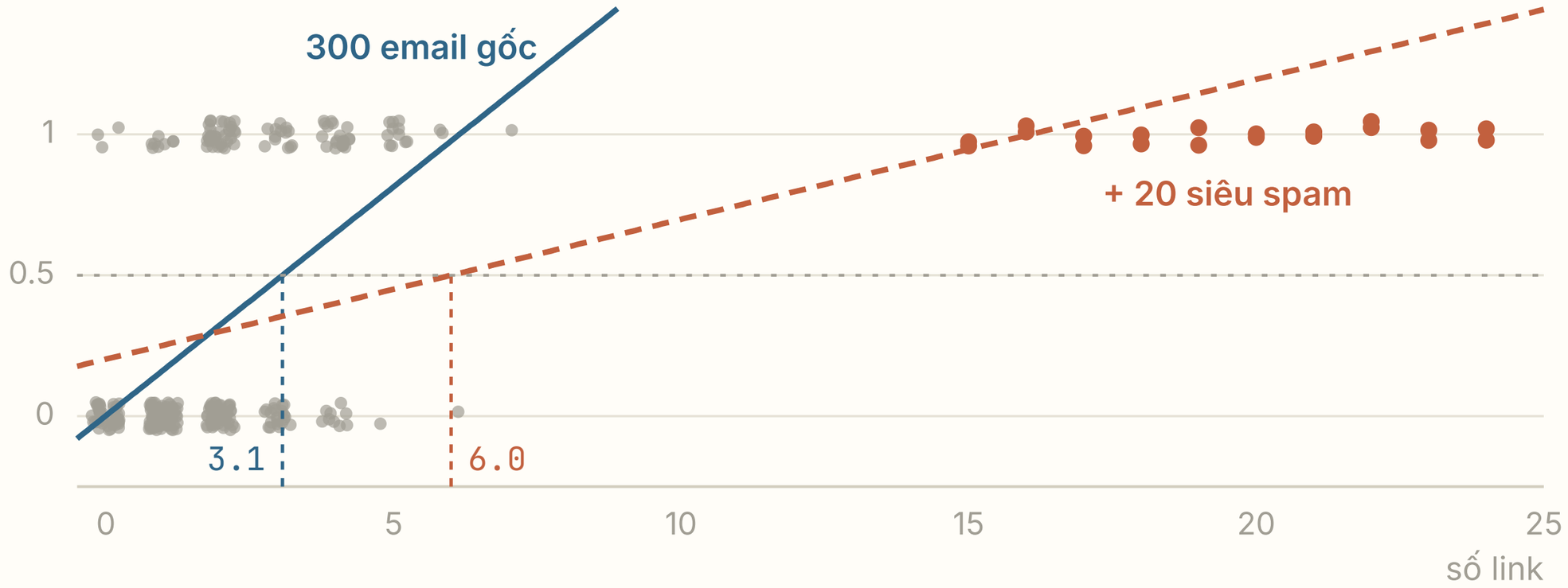
# Bị điểm dễ kéo lệch

Chỉ dùng số link. Thêm 20 email “siêu spam” có 15–24 link.

| | Ngưỡng | Accuracy test |
|---|---|---|
| 300 email gốc | 3.07 link | 0.78 |
| Thêm 20 siêu spam | **6.00 link** | **0.68** |

Hàm bình phương phạt cả chỗ **đúng mà quá chắc chắn**.

# Ngưỡng dịch từ 3 lên 6 link




Linear regression trên nhãn 0/1: thêm 20 email siêu spam làm đường thoải xuống, ngưỡng 0.5 dịch từ 3.1 lên 6 link.

HỎI NHANH

## Thêm vào train 20 email có từ 15 đến 24 link, email nào cũng là spam thật. Vì sao ngưỡng của linear regression lại dịch từ khoảng 3 link lên 6 link?

A Email dễ, đã đúng nhãn, vẫn bị phạt vì dự đoán vượt quá 1

B Tỉ lệ spam trong train tăng nên ngưỡng phải tăng theo

C 20 email ấy thật ra bị gán nhãn sai nên làm hỏng mô hình

D Mô hình bị overfit vì có thêm mẫu ở vùng xa

02

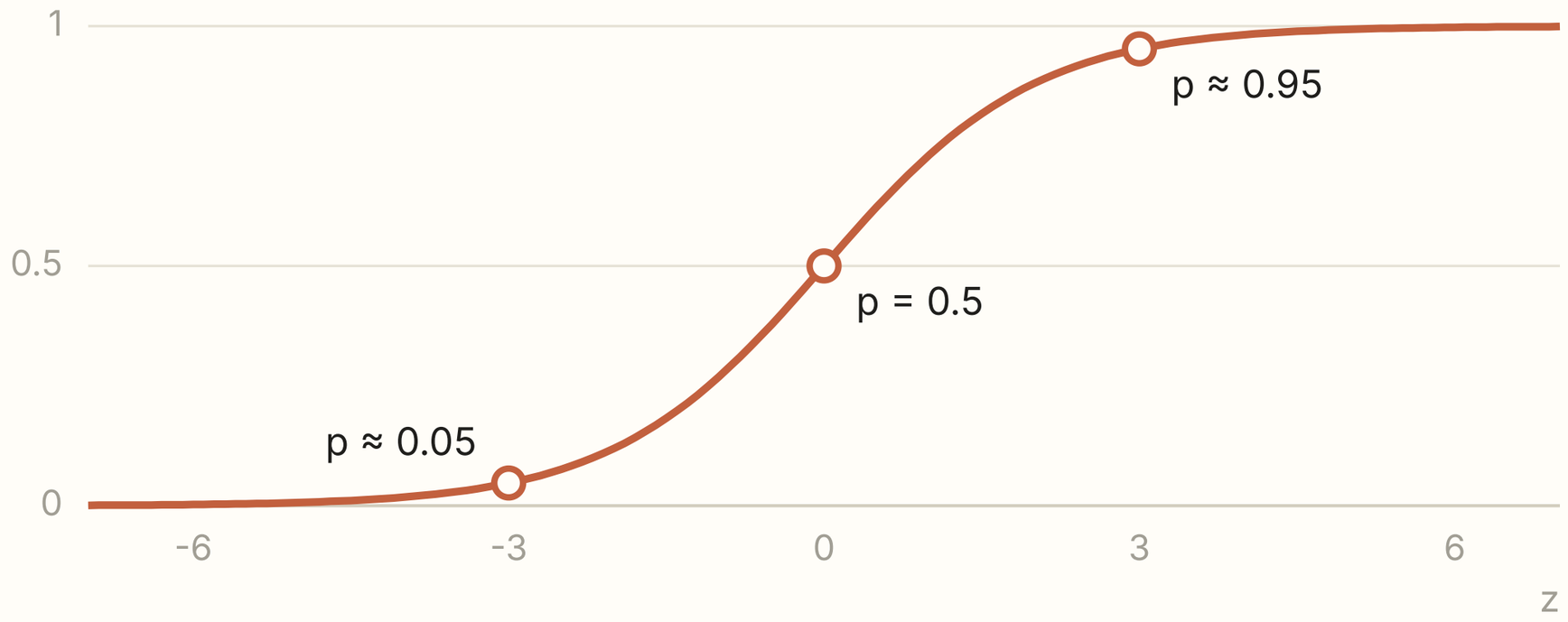
# Mô hình

# Sigmoid

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w}^\top \mathbf{x} + b$$

- Mọi số thực vào, một số trong $(0, 1)$ ra.
- $z = 0 \Rightarrow p = 0.5$; $z = 3 \Rightarrow p \approx 0.95$.

# Từ z sang xác suất




Sigmoid nhận mọi số thực z và trả về một xác suất trong (0, 1).

# Log-odds

$$z = \log \frac{p}{1-p}$$

Odds $\frac{p}{1-p}$: "mấy ăn một". $p = 0.75$ thì odds bằng $3$.

Tuyến tính cho **log-odds**, không phải cho xác suất.

Tăng $x_j$ một đơn vị: odds nhân $e^{w_j}$.

HỎI NHANH

Mô hình cho một email $p = 0.8$. Odds spam của email ấy bằng bao nhiêu?

A 4

B 0.8

C 0.2

D 1.25

# Ranh giới quyết định

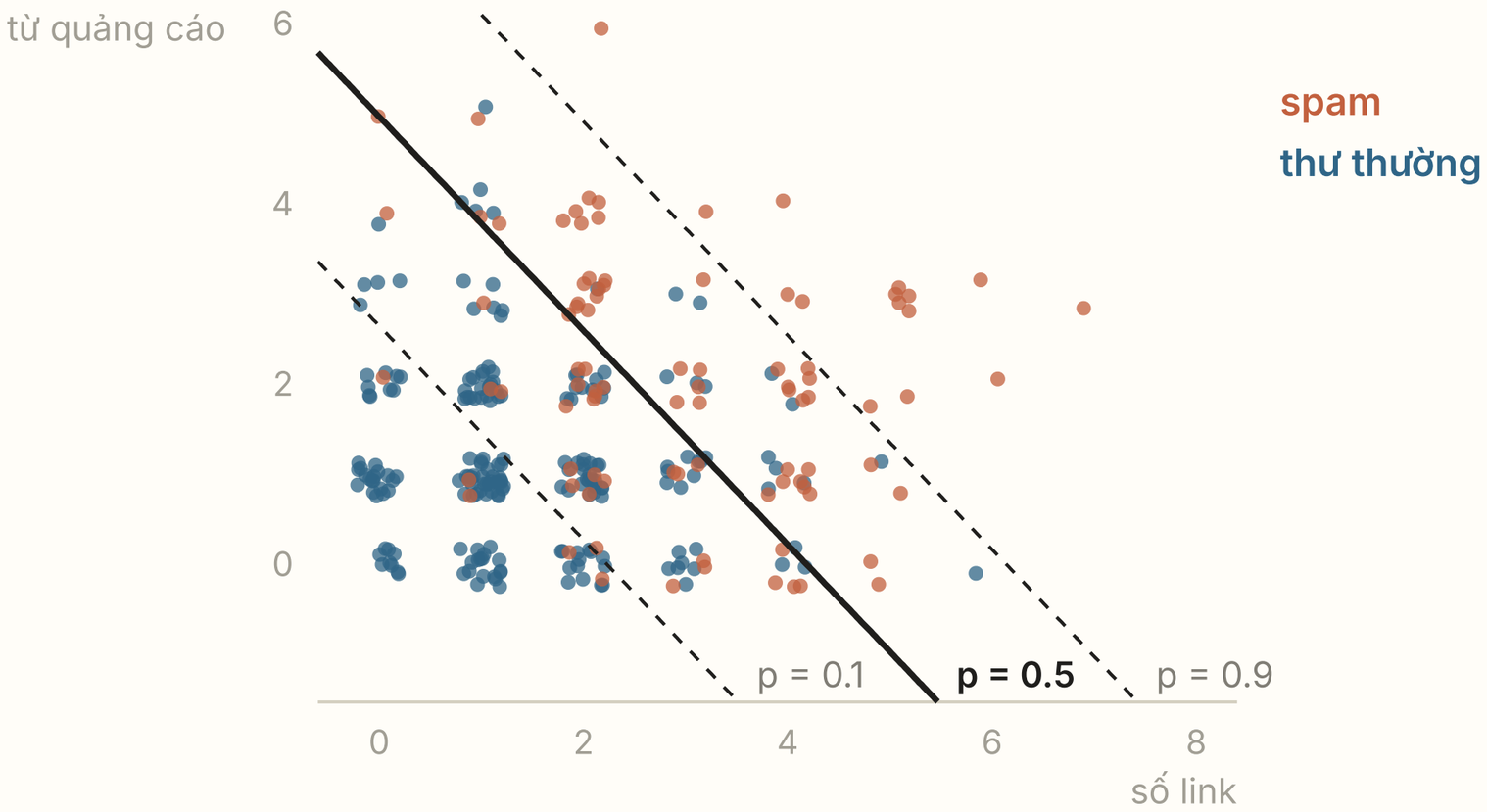
$p \geq 0.5 \iff z \geq 0$, nên ranh giới là

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

Hai đặc trưng: một **đường thẳng**. Sigmoid không làm ranh giới cong.

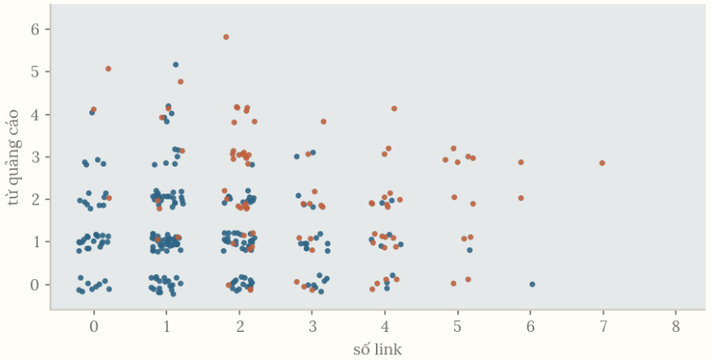
Muốn cong: thêm $x_1^2, x_2^2, x_1 x_2$ như bài 2.1.

# Ba mức xác suất

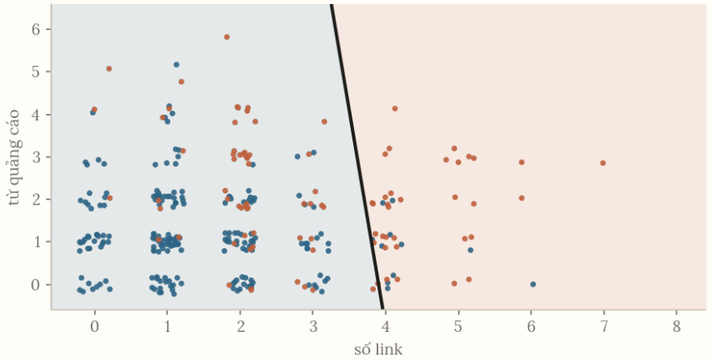



Các mức xác suất của logistic regression là những đường thẳng song song; p = 0.5 là ranh giới.

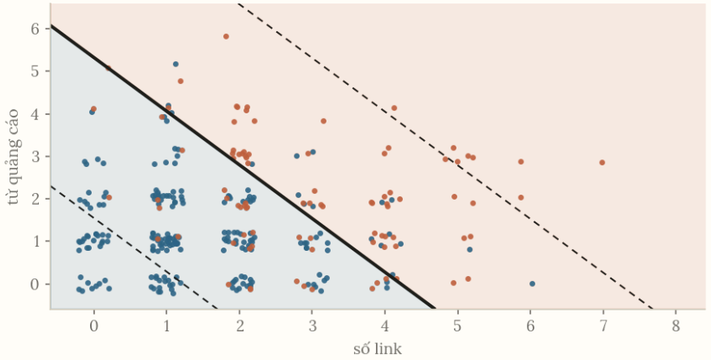
# Ranh giới dịch theo từng vòng



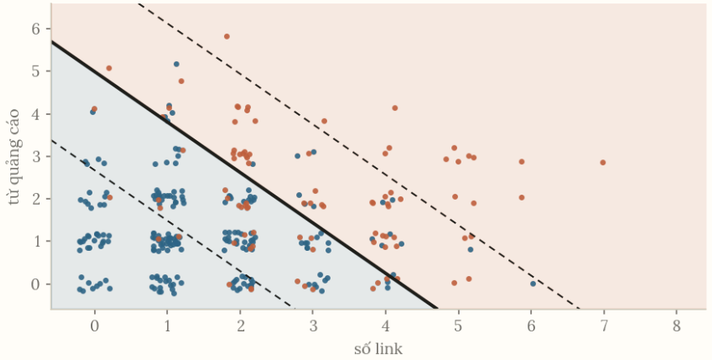

vòng 1: loss 0.6751; w = (-0.02, -0.04), b = -0.10




vòng 10: loss 0.5903; w = (0.17, 0.02), b = -0.68




vòng 100: loss 0.4203; w = (0.73, 0.58), b = -3.10




vòng 5000: loss 0.3995; w = (1.12, 0.95), b = -4.73

Gradient descent của bài trên 300 email: ranh giới p = 0.5 (nét liền) và hai mức 0.1, 0.9 (nét đứt) dịch dần về chỗ tách spam.

# Thí nghiệm siêu spam, lần nữa

```
m = LogisticRegression(C=np.inf).fit(L, yy)
t = -m.intercept_[0] / m.coef_[0, 0]
```

| | Ngưỡng | Accuracy test |
|---|---|---|
| 300 email gốc | 3.01 link | 0.78 |
| Thêm 20 siêu spam | 3.01 link | 0.78 |

Ngưỡng không nhúc nhích.

Email 24 link đã có $p \approx 1$: đúng và tự tin đúng thì gần như không bị phạt.

Lý do nằm ở hàm mất mát.

HỎI NHANH

Logistic regression một feature có $w = 2, b = -6$. Ranh giới quyết định nằm ở đâu?

A $x = -3$

B $x = 6$

C $x = 0.5$

D $x = 3$

# 03

# Cross-entropy

# Từ maximum likelihood

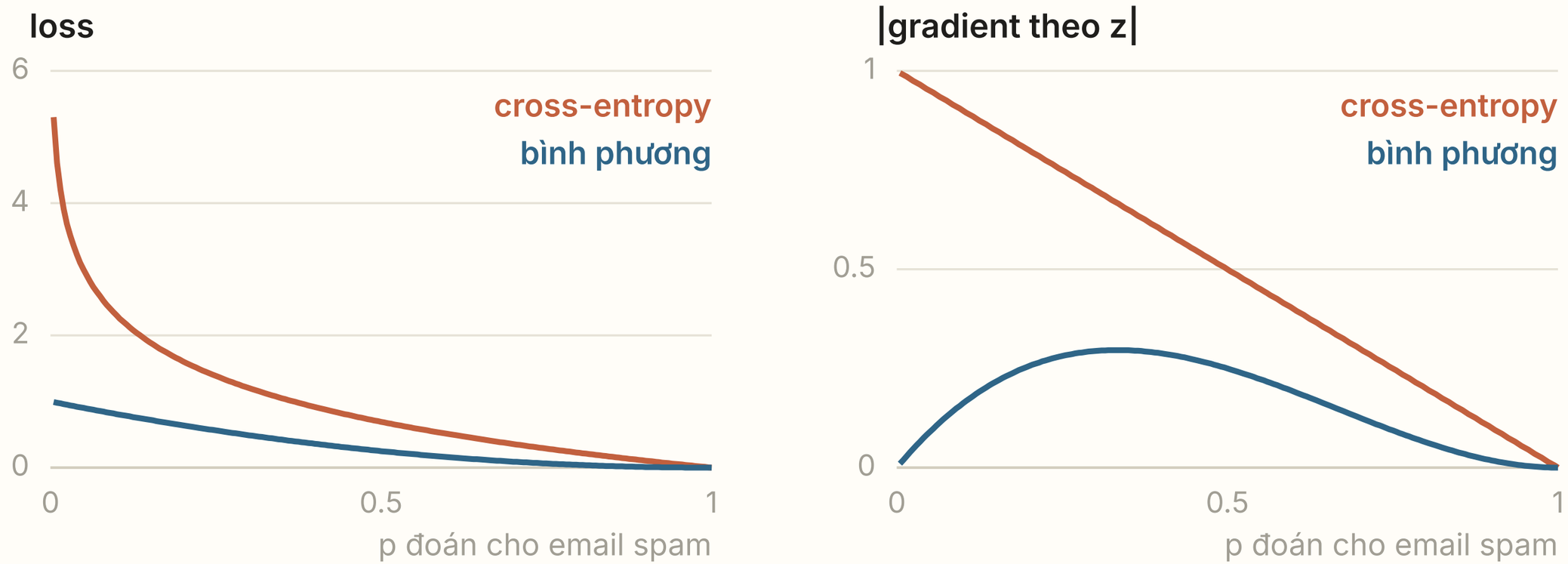
Xác suất mô hình gán cho nhãn đã xảy ra: $p_i^{y_i}(1-p_i)^{1-y_i}$

Nhân lại, lấy log, đổi dấu, chia $n$:

$$J = -\frac{1}{n}\sum_i \Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big]$$

Spam thật: đoán $0.99$ mất $0.01$ · đoán $0.5$ mất $0.69$ · đoán $0.01$ mất $4.6$

# Gradient của bình phương và cross-entropy




Với email spam bị đoán p nhỏ: cross-entropy phạt nặng và cho gradient lớn; bình phương cho gradient gần 0.

Spam thật, đoán $p = 0.01$: gradient theo $z$ là $-0.0196$ (bình phương) so với $-0.99$ (cross-entropy).

HỎI NHANH

Với $p = \sigma(z)$, nếu dùng bình phương sai số thay cho cross-entropy, thì khi mô hình rất tự tin mà sai, chuyện gì xảy ra?

A Gradient theo $z$ vẫn luôn bằng $p - y$

B Bình phương sai số không tính được với nhãn 0 và 1

C Gradient theo $z$ có thể rất nhỏ, do thừa số $p(1-p)$

D Không có gradient, vì sigmoid không có đạo hàm

04

# Gradient và tự cài

# Đạo hàm

$\sigma'(z) = p(1-p)$. Theo quy tắc chuỗi:

$$\frac{\partial \ell}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) = p - y$$

$$\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\mathbf{p} - \mathbf{y})$$

Giống linear regression: chỉ thay $\hat{\mathbf{y}}$ bằng $\mathbf{p}$.

| Đại lượng | Shape |
|---|---|
| `X` | $(300, 2)$ |
| `z = X @ w + b` | $(300,)$ |
| `p = sigmoid(z)` | $(300,)$ |
| `X.T @ (p - y) / n` | $(2,)$ |

# Cài đặt và kiểm gradient

```
def loss(theta, X, y):
    logits = X @ theta[:-1] + theta[-1]
    return np.mean(np.logaddexp(0., logits)
                   - y * logits)

def grad(theta, X, y):
    r = sigmoid(X @ theta[:-1] + theta[-1]) - y
    return np.append(X.T @ r / len(y), r.mean())
```

Loss tính thẳng từ logit bằng `logaddexp`: không tràn số, không cần kẹp, nên gradient đúng là $p - y$.

So với sai phân trung tâm:

KẾT QUẢ

```
sai số tương đối: 5.8e-12
```

Gradient descent, lr $= 0.5$, 5000 vòng:

KẾT QUẢ

```
w = [1.124 0.947]   b = -4.73
loss ban đầu: 0.6931   sau 5000 vòng: 0.3995
```

# Đối chiếu scikit-learn

KẾT QUẢ

```
scikit-learn: w = [1.124 0.947]   b = [-4.73]
trùng gradient descent: True
```

- Quy luật thật: $(1.1,\ 0.9,\ -4.5)$.
- scikit-learn dùng L-BFGS: vài chục bước thay vì vài nghìn.
- Không có nghiệm đóng.

THAM SỐ C

Mặc định `C=1.0`: có phạt L2, `C` tỉ lệ nghịch với $\lambda$.

`C=np.inf` tắt phạt để so với code tự viết.

Dữ liệu tách được hoàn toàn mà không phạt: trọng số tăng mãi.

HỎI NHANH

Khởi tạo mọi tham số bằng 0 cho một bài phân loại hai lớp. Cross-entropy lúc đầu bằng bao nhiêu?

A $0$

B $0.5$

C $\log 2 \approx 0.693$

D Không xác định được

05

# Đọc kết quả

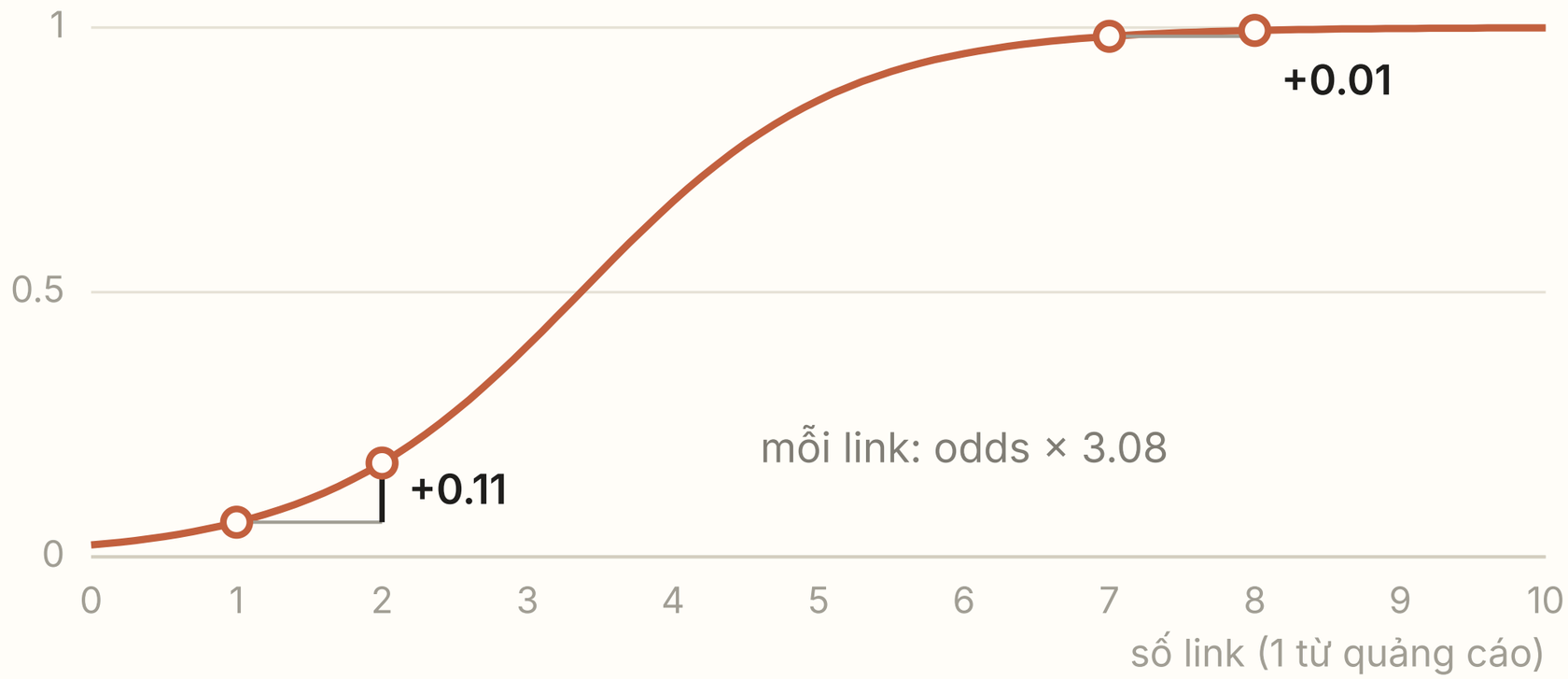
# Đọc hệ số qua odds

Thêm 1 link: odds $\times 3.08$. Thêm 1 từ quảng cáo: odds $\times 2.58$.

| (số link, số từ) | P(spam) |
|---|---|
| (1, 1) | 0.065 |
| (2, 1) | 0.177 |
| (7, 1) | 0.983 |
| (8, 1) | 0.995 |

- Odds nhân đều $\times 3$ mỗi link.
- Xác suất thì không: $1 \to 2$ link tăng $0.11$ ; $7 \to 8$ link chỉ tăng $0.01$.
- Odds nhân đều, xác suất bão hòa.

# Odds nhân đều, xác suất bão hòa




Mỗi link nhân odds lên khoảng 3 lần, nhưng xác suất tăng nhiều ở giữa và gần như đứng yên khi đã gần 1.

HỎI NHANH

Một email đang có $p = 0.5$. Hệ số của "số link" là $w = \ln 3$. Thêm một link thì $p$ thành bao nhiêu?

A $1.5$

B $0.83$

C $0.6$

D $0.75$

# Ngưỡng quyết định

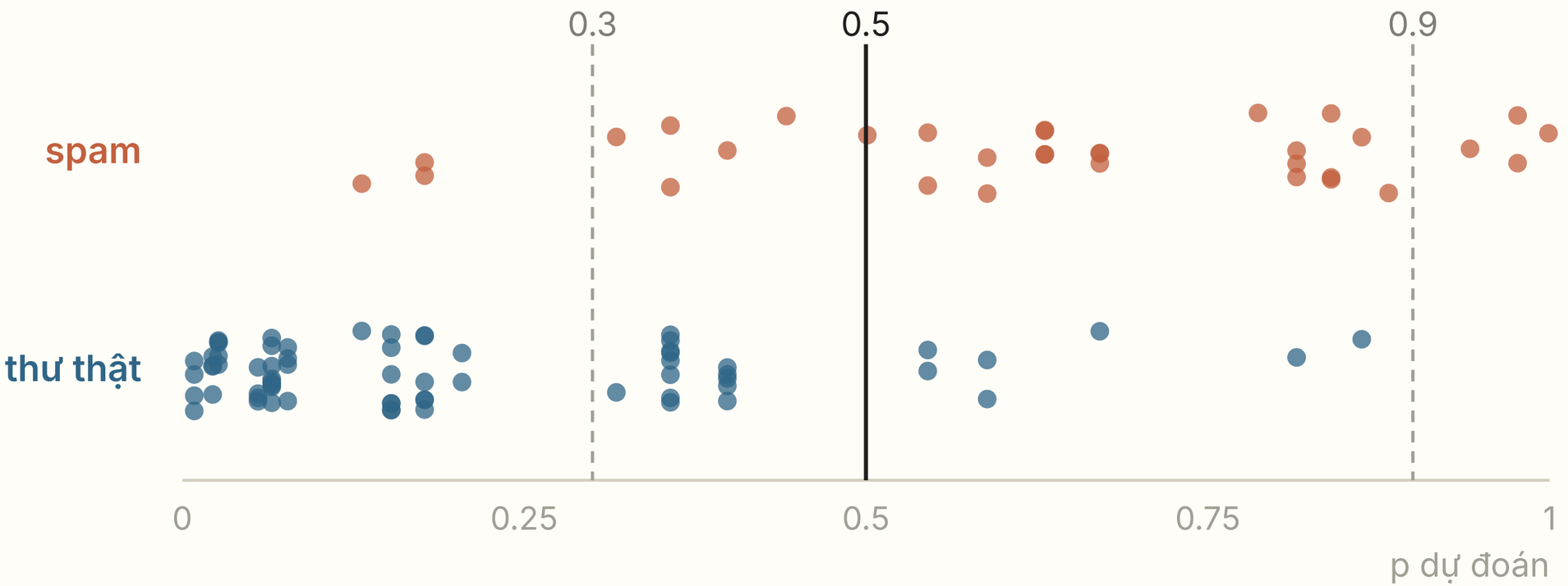
100 email test, 33 spam. Bảng này chỉ để thấy sự đánh đổi; ngưỡng dùng thật phải chọn trên validation.

| Ngưỡng | Chặn đúng spam | Chặn nhầm thư thật | Lọt spam |
|---|---|---|---|
| 0.3 | 30 | 21 | 3 |
| 0.5 | 25 | 7 | 8 |
| 0.9 | 4 | 0 | 29 |

Accuracy $0.85$; luôn đoán “không spam”: $0.67$.

- `predict` cắt ở $p > 0.5$; đúng bằng $0.5$ thì scikit-learn chọn lớp 0.
- Ngưỡng là quyết định nghiệp vụ, theo cái giá của từng loại sai.
- So accuracy với mốc **đoán lớp đông nhất**.
- Log loss: mốc $0.635$, mô hình $0.371$.

# 100 email test xếp theo p




100 email test theo xác suất dự đoán: email bên phải ngưỡng bị chặn; dời ngưỡng là đổi số thư thật bị chặn nhầm lấy số spam lọt.

HỎI NHANH

## Với bộ lọc thư rác, chặn nhầm một thư thật tốn kém hơn nhiều so với để lọt một thư spam. Ta nên chỉnh ngưỡng thế nào?

A Hạ ngưỡng xuống $0.3$ để chặn được nhiều spam hơn

B Nâng ngưỡng lên, và chọn mức cụ thể trên validation

C Giữ $0.5$, vì đó là ngưỡng chuẩn của mô hình

D Huấn luyện thêm nhiều vòng cho mô hình chắc hơn

06

# Nhiều lớp: softmax

# Softmax regression

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$$

- Loss: $-\log p_y$. Gradient theo logits: $\mathbf{p} - \mathbf{e}_y$.
- $K = 2$: $p_1 = \sigma(z_1 - z_2)$, chính là logistic regression.

| Đại lượng | Shape |
|---|---|
| `X` | $(1300, 64)$ |
| `W, b` | $(64, 10)$, $(10,)$ |
| `P = softmax(X @ W + b)` | $(1300, 10)$ |
| `X.T @ (P - Y) / n` | $(64, 10)$ |

# Trừ max trước khi lấy mũ

$e^{1000}$ tràn số: `inf / inf = nan`.

Cộng cùng hằng số vào mọi $z_k$ không đổi softmax, nên trừ $\max_k z_k$.

```
def softmax(Z):
    Z = Z - Z.max(axis=-1, keepdims=True)
    E = np.exp(Z)
    return E / E.sum(axis=-1, keepdims=True)
```

KẾT QUẢ

```
[0.665 0.245 0.09 ]
[nan nan nan] [0.665 0.245 0.09 ]
```

Tính $\log p_k$ thì dùng log-sum-exp:

$$\log p_k = z_k - m - \log \sum_j e^{z_j - m}$$

# Chữ số viết tay

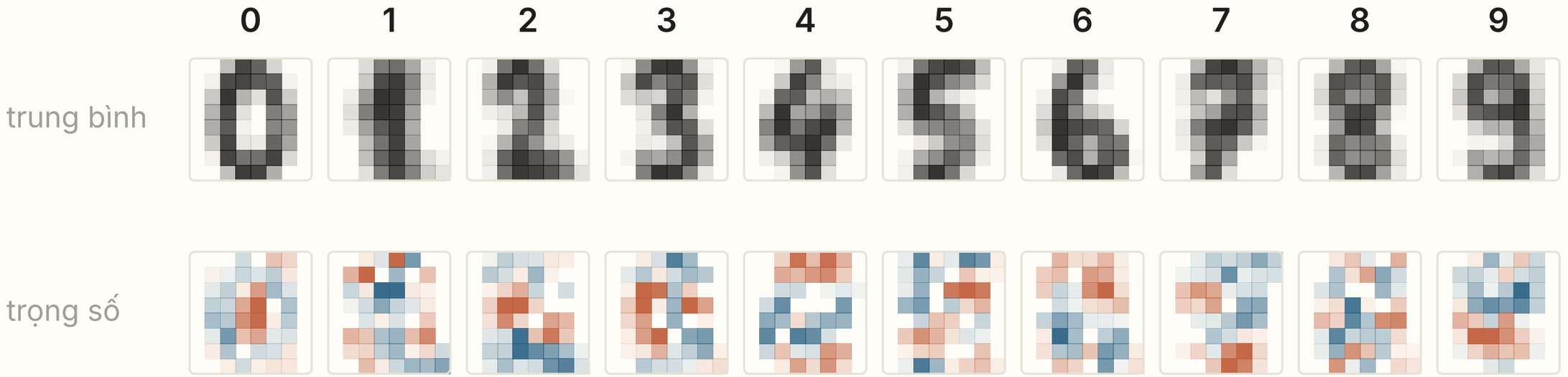
1797 ảnh $8 \times 8$, 1300 để học, 497 để kiểm tra.

Gradient descent 5000 vòng, phạt L2 khớp `LogisticRegression(C=1)`.

| | Accuracy test |
|---|---|
| Tự cài | 0.978 |
| scikit-learn | 0.978 |

Nhãn trùng trên mọi ảnh test; xác suất lệch tối đa $0.032$ vì gradient descent chưa hội tụ hẳn.

# Mô hình nhìn gì




xanh: nét mực ở đây tăng điểm lớp · cam: giảm điểm

Hàng trên: ảnh trung bình của mỗi chữ số. Hàng dưới: 64 trọng số của lớp đó xếp thành ảnh 8 × 8.

HỎI NHANH

Cộng thêm 100 vào mọi logit $z_1, \ldots, z_K$. Xác suất softmax thay đổi thế nào?

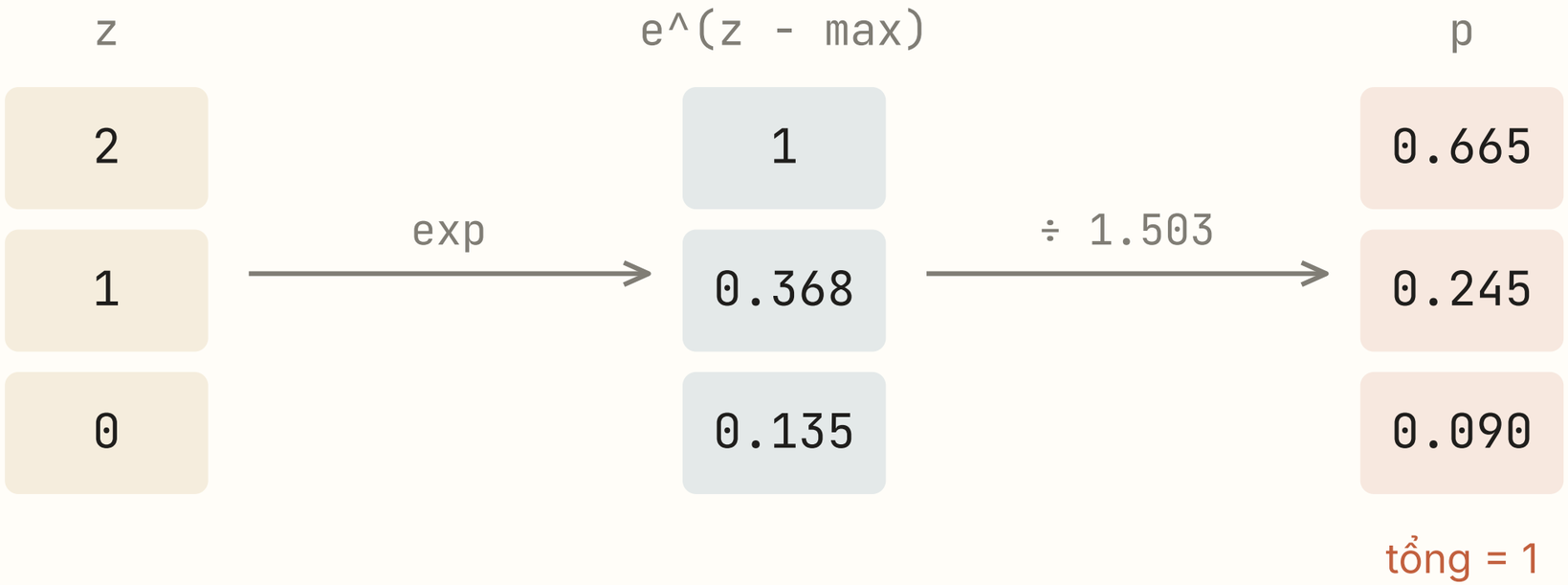
A Không thay đổi

B Lớp có logit lớn nhất tăng lên gần 1

C Mọi xác suất đều tăng lên

D Thành nan

# Softmax giữ kết quả sau khi trừ max




Softmax của (2, 1, 0): trừ max, lấy mũ, chia cho tổng.

# Tính tay trên lớp

$x = 2$, $y = 1$, $w = 1$, $b = -1$. Tính $p$, loss, gradient theo $w$.

Softmax của $\mathbf{z} = (2, 1, 0)$?

$z = 1$, $p = 0.731$, loss $= 0.313$, gradient $= (0.731 - 1) \cdot 2 = -0.538$.

Trừ max: $(0, -1, -2)$ → mũ: $(1,\ 0.368,\ 0.135)$ → chia tổng $1.503$:

$$\mathbf{p} = (0.665,\ 0.245,\ 0.090)$$

# Bẫy cài đặt

| Bẫy | Hậu quả | Cách tránh |
|---|---|---|
| $\log 0$ khi $p$ làm tròn thành 0 hoặc 1 | Loss `inf` hoặc `nan` | Kẹp $p$ vào $[\varepsilon, 1-\varepsilon]$ |
| `np.exp` trên logit lớn | Softmax `nan` | Trừ max, log-sum-exp |
| Nộp `predict` cho metric log loss | $0.371 \to 5.407$ | Nộp `predict_proba` |
| Đặc trưng lệch thang đo | Hội tụ chậm, phạt không đều | `StandardScaler` trong pipeline |

HỎI NHANH

# Một mô hình khác đoán đúng 196 trên 200 email, nhưng lại nộp nhãn 0 và 1 cho đề chấm bằng log loss. Điểm log loss khoảng bao nhiêu?

A Khoảng $0.02$, đúng bằng tỉ lệ sai

B Khoảng $0.72$, tệ hơn cả cách đoán hằng số

C Khoảng $0.07$, vì đúng tới 98%

D Vô cùng, vì có email bị đoán sai

# Khi đi thi

- **Log loss**: nộp xác suất theo scorer, kiểm thứ tự lớp và tổng xác suất; đánh giá cả hiệu chỉnh xác suất.
- **F1, lớp hiếm**: chọn ngưỡng trên validation; thử `class_weight="balanced"`.
- **Văn bản**: TF-IDF + logistic regression là baseline nhẹ để đối chiếu (bài 6.1).
- **Dữ liệu bảng**: `make_pipeline(StandardScaler(), LogisticRegression(C=...))`, dò `C` trên thang log.
- **Stacking**: có thể dùng logistic regression ở tầng cuối với OOF predictions (bài 8.4).

# Tóm tắt

- Tuyến tính cho log-odds: $p = \sigma(\mathbf{w}^\top \mathbf{x} + b)$; mỗi đơn vị của $x_j$ nhân odds $e^{w_j}$.
- Cross-entropy từ maximum likelihood; lồi; gradient $\frac{1}{n} X^\top (\mathbf{p} - \mathbf{y})$.
- Ranh giới vẫn thẳng; ngưỡng chọn theo cái giá của từng loại sai.
- Softmax cho $K$ lớp, gradient $\mathbf{p} - \mathbf{e}_y$; trừ max; tính loss ổn định từ logits.

07

# Luyện đề

LUYỆN ĐỀ 1

# Mô hình lọc spam cho ra logit $z$, qua sigmoid thành $p$. Ta nên huấn luyện bằng loss nào, và vì sao?

A Bình phương $(y-p)^2$, vì dễ hiểu, coi phân loại như hồi quy

B Hinge $\max(0, 1-yz)$, vì nó tối đa lề và chống được spam cố ý lách

C Trị tuyệt đối $|y-p|$, vì ít nhạy với nhãn bị gán nhầm

D Cross-entropy, vì gradient theo $z$ là $p-y$, mạnh khi sai

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 273

LUYỆN ĐỀ 2

# Bài phân loại hai lớp có ba feature: tuổi từ 18 đến 80, thu nhập từ 20 đến 500 triệu, và điểm tín dụng từ 300 đến 850. Logistic regression không chuẩn hóa đạt accuracy 72%, thêm `StandardScaler` thì đạt 84%. Vì sao?

A Chuẩn hóa loại bỏ các giá trị thu nhập bất thường, chống overfit nhóm giàu

B Chuẩn hóa đưa dữ liệu về quanh 0, vào vùng gần tuyến tính của sigmoid

C Cùng thang đo thì tối ưu hội tụ tốt, cột thu nhập không còn lấn át gradient

D Chuẩn hóa tăng learning rate cho feature ít biến thiên nhất

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 305

LUYỆN ĐỀ 3

Feature nhị phân $x_1$ rất hiếm: trong train, email nào có $x_1 = 1$ cũng là spam. Logistic regression không có regularization. Trọng số tối ưu $\hat{w}_1$ sẽ ra sao?

A Hữu hạn, bằng log-odds spam khi $x_1 = 1$

B Bằng 0, vì feature quá hiếm nên không mang thông tin

C Tăng mãi, vì gradient theo $w_1$ không bao giờ về 0

D Âm, để bù cho việc feature hiếm khi bật lên

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q2.3(c)

# Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: thêm đặc trưng $x_1^2$, $x_2^2$, $x_1 x_2$ cho dữ liệu spam, xem ranh giới và log loss thay đổi thế nào.